
Documentation for VCU118 card used for APARAJIT

GitHub Repository : https://github.com/ANDnwl-IITB/Board_git_files_NWL

In this repository for VCU118 only three directories are utilized :

1. VCU118_Aparajit 2. VCU118_Aparajit_bitfiles 3. TCL_SCRIPTS/vcu118_aparajit.tcl

I.] VCU118_Aparajit

1. **component** :- component with interfaces (toplevel)

2. **edn** :-

- -glue.ooc – this directory have the other necessary edn files required.
- -because processor_1x1x32.edn & engine.core.edn file were too large , they could not be uploaded in repository.
- -latest processor and engine edn files are kept in madhav system (lab server) which are used for latest bitstream generation.

3. **hsys** :- Used directories:

- - nic_subsystem is used for ethernet
- - acb_afb_complex needed including NIC tap

4. **ip** :- contains all the ips used

5. **verilog** :- contains verilog files of some units used in design

6. **vhdl & additional vhdl**:- VHDL files of design units.

7. **vhdl_libs**:- libraries required

8. **constraints**:- have the constraints (.xdc file)

9. **TESTS**:- Ethernet_tests/testingNIC_1.2 have ethernet tests and inside that host directory have python tests such as ping, throughput etc.
ddr4_tests/marches have the marches tests to test memory subsystem

10. **vivado_synth** :- Contains compile.tcl for bitstream generation of APARAJIT.

11. **docs** :- 1p_1e figure + relevant docs.

II.] VCU118_Aparajit_bitfiles: contains the vcu118 latest bitfile inside bitfile_<latest_date> directory.

III.] TCL_SCRIPTS/vcu118_aparajit.tcl : contains TCL scripts for programming VCU118 card.

Bitstream Generation for VCU118 : IMP NOTE: It can be done only in madhav system (lab server : 10.107.90.220) for VCU118 Aparajit

1. ssh into madhav system
 2. git clone the repository (only if not present there at desktop)
 3. Enter inside Board_git_files_NWL/VCU118_Aparajit/edn/
 4. EDN files will be needed for bistream generation so follow strictly:-
 - (i) Make a directory named " new_<date> " [useful for identifaction of latest edn].
 - (ii) Upload latest processor_1x1x32.edn & engine_core.edn inside this directory provided by MPD sir.
- // because processor_1x1x32.edn & engine_core.edn file were too large , they could not be uploaded in GitHub repository.
5. After successful upload of edn now enter the directory VCU118_Aparajit/vivado_synth and look for compile.tcl
 6. Edit the compile.tcl file
– example : read_edif \$EDN/new_24aug/processor_1x1x32.edn
read_edif \$EDN/new_24aug/engine_core.edn
NOTE: It needs to be edited according to your ../edn/new_<date>/ directory
 7. Now run command ./run.sh inside vivado_synth.
 8. Bitstream will get generated as .bit and .ltx file .
 9. Now make a directory inside Board_git_files_NWL/VCU118_Aparajit_bitfiles/ naming bitfile_<generated_date>
 10. Copy the newly generated .bit and .ltx from Board_git_files_NWL/VCU118_Aparajit/vivado_synth/ to this directory and git push.
- // The latest bitfile identifcation will be according to dates in directory.

TCL_SCRIPTS EDITING FOR PROGRAM AND RESET

1. Goto Board_git_files_NWL/vcu118_aparajit.tcl/
 2. Change bitfile location from both program_vcu118.tcl and reset_vcu118.tcl whenever bitfile gets updated.
 3. Run this tcl script only in nwl_3 system (currently in Research Park Powai Labs) // IMP POINT
 4. If need to run in other systems, change the path of Xilinx Vivado according to that system in program.sh and reset.sh
-

RUNNING TESTS : It will be done in nwl_3 because VCU118 card is connected to this system and it is kept at Powai Labs , Research park IITB

MARCHES TEST

1. Enter into AjitToolChain/ajit-toolchain/Board_git_files_NWL/VCU118_Aparajit/TESTS/ddr4_tests/marches/marches_dram_full — repo was git cloned inside AjitToolChain

2. Check main.c and config.yaml. [Example : clk frequency in config.yaml]

3. Now open AjitToolChain/ajit-toolchain/docker/ajit_build_dev

4. Export UART commands :

(i) Type command "export AJIT_DEBUG_UART=/dev/ttyUSB" press Tab twice

(ii) You'll get to see ports USB0 USB1 USB2.

(iii) Now remove serial UART from the USB hub and repeat the step (i)

(iv) You'll get to see only two ports (either USB0 USB1 or USB1 USB2)

(v) **CASE A/** If you see USB0 USB1 then your serial UART is USB2 and add serial UART in USB hub again.

Take debug UART as higher index in remaining ports other than USB2 i.e. USB1.

Type commands: "export AJIT_DEBUG_UART=/dev/ttyUSB1" Enter

"export AJIT_SERIAL_UART=/dev/ttyUSB2" Enter

CASE B/ If you see USB1 USB2 then your serial UART is USB0 and add serial UART in USB hub again.

Take debug UART as higher index in remaining ports other than USB0 i.e. USB2

Type commands: "export AJIT_DEBUG_UART=/dev/ttyUSB2" Enter

"export AJIT_SERIAL_UART=/dev/ttyUSB0" Enter

(vi) Run command "./run_with_uarts.sh"

5. Enter the docker : "./attach_shell.sh" and enter

6. Inside docker type commands: "source set_ajit_home" enter
"source ajit_env"

7. Run command "cd

Board_git_files_NWL/VCU118_Aparajit/TESTS/ddr4_tests/marches/marches_dram_full"

8. Now open minicom in another terminal running command : "sudo minicom -s"

9. Do Serial UART setup in minicom

a. Serial Port Setup.

b. Press A and change serial port to /dev/ttyUSB<serial_UART> and press Enter. —serial uart can be 0,2

c. Exit

10. Open Board_git_files_NWL/TCL_SCRIPTS/vcu118_aparajit_tcl/ in new terminal.

11. Run command ./program.sh — Program the VCU118 card with latest bitfile.

12. Wait till it get programmed and then go back docker shell and run command `"/run.sh /dev/ttyUSB<debug_uart>"` —debug uart can be 1,2
 13. Run command `"r mode"` . It should return 0:0
 14. Run command `"s run.script"` .
 15. Output of marches will be there in minicom terminal after the run.script is completed successfully.
 16. If you want to repeat test you can run command `"/reset.sh"` in terminal where `Board_git_files_NWL/TCL_SCRIPTS/vcu118_aparajit_tcl/` is opened and repeat from step 12.
-

ETHERNET TEST ADDITIONAL STEPS

1. After running command `"s run.script"` for Ethernet tests inside docker shell :
`Board_git_files_NWL/VCU118_Aparajit/TESTS/Ethernet_tests/testingNIC_1.2/`
 2. Open another terminal and go into same directory outside shell `testingNIC_1.2/host`
 3. Run python tests in this directory: run command `"sudo python3 ping.py"` .
 4. Output will be there for the ping test . Also `throughput.py` , `reliable_image_send.py` etc. tests can be ran in same way.
-